

MARI Quadcopter Drone

Connor Lee Prior
Department of Engineering
CSU East Bay
connorprior2000@gmail.com

Christopher Ruiz-Guerra
Department of Engineering
CSU East Bay
cruizguerra@horizon.csueastbay.edu

Nahom Solomon
Department of Engineering
CSU East Bay
nsolomon@horizon.csueastbay.edu

Seth Iris Canonigo
Department of Engineering
CSU East Bay
scanonigo@horizon.csueastbay.edu

Abstract

The MARI drone group developed subsystems to work together and achieve a flight controller. We've created software that reads data from a BNO055 Inertial measurement unit (IMU) and a BMP39 to extract Roll, Pitch, Yaw, temperature and relative altitude via I2C lines. We selected to use a STM32F756BG microcontroller running on the maximum speed of 216 MHz in conjunction with a Proportional-Integral-Derivative (PID) control loop. This system allows real time adjustments based on the user's PWM outputs from a radio transmitter and outputs PWM signals driving our Electronic Speed Controllers to our motors. We developed a Printed Circuit Board (PCB) which has three dedicated and separate power domains, utilizes a switching regulator to lower 14.8V to a 5V domain and a Low-Dropout regulator to drop 5V to 3.3V. We then utilized a long-range radio (LoRa) module to wirelessly transmit data over a kilometer which communicates with the microcontroller over a simple UART connection using Attention commands (AT). This allows us to see data from the IMU and BMP390 & PID coefficients in real time. We've created a MARI LoRa Monitor which uses a PyQt6 framework to parse through the LoRa data packet and updates a live Graphical User Interface (GUI) with telemetry of PID coefficients. Despite the lack of stable flight, we have validated power in all domains, and established PID communication on the three axes. We've overcome catastrophic hardware failures, including the burning three boards. These failures helped the team learn the value of power system planning, circuit design, hardware bring-up, hardware validation, firmware programming on microcontrollers, we all got experience in PCB and schematic design and lastly, the value of working together as a team.

Introduction

For our Senior Design project we decided to create a quadcopter drone. This project posed multiple design challenges that helped us better

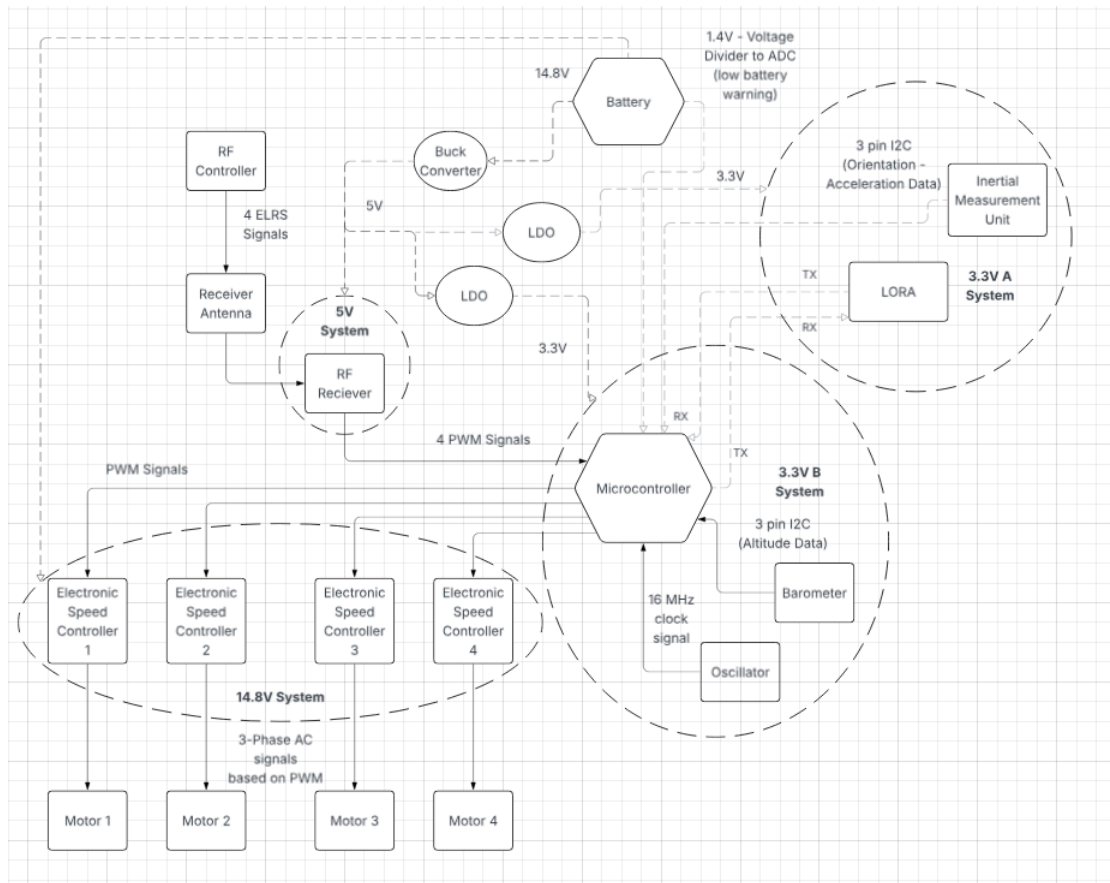
understand both the circuit board design and programming needed to build a drone that can actually fly. Instead of using a prebuilt system that we could just plug in and use out of the box, we had to design our own printed circuit board. This introduces problems for power distribution for sensitive components, interpreting communication signals through protocols such as I2C and UART, wirelessly transmitting data into a human-readable format, and choosing the right parts that are able to meet the design criteria we set out to reach.

As a flying quadcopter is an inherently unstable system, another big component of this project was stabilizing the drone by making high frequency adjustments to the motors to maintain the drone's position. This involves using a Proportional-Integral-Derivative control loop to make microadjustments to the outputs to the motors.

In this article we will describe our process for creating our quadcopter drone, the problems we encountered when designing it, and how we solved them.

Hypothesis

When looking at features we wanted for our quadcopter drone, the main things we were looking for was for it to be remote controlled, self-balancing, and for the user to be able to add any additional modules onto the drone if they wanted to. To do this, we first needed a microcontroller, so we could control our propellers with computer programming. For our remote control, we used a radio controller which would send radio signals from the user to our receiver on the drone. For the drone to self-balance, we needed to add an Inertial Measurement Unit (IMU). This allows us to detect the angular acceleration of the drone in accordance with pitch, roll, and yaw, which is very important to stabilizing the drone. We also added a barometer to be able to set conditions that would limit the height the drone could fly at. For telemetry, we used a long-range radio (LoRa) module to wirelessly transmit data to us mid-flight. Finally, we made sure we had a lot of extra pins we could access on the microcontroller so we could add additional components later down the line.



Block Diagram

Our project has a few different subsystems that work together to get our drone flying. The first of these is the microcontroller, the brain of the system. We had chosen the STM32F756BG as our microcontroller. By programming the microcontroller, we can use that logic to control the outputs to our motors based on both the user inputs and the data from the IMU. We also decided to hook our microcontroller up to a crystal oscillator to speed up the clock frequency of our microcontroller to the maximum speed of 216 MHz.

The main two sensors we used in the project were the BNO055 IMU and the BMP390 Barometer. The main purpose of the IMU is to send acceleration and position data of the drone to the microcontroller. To keep track of linear acceleration, the IMU has an accelerometer. The accelerometer is a MEMS (Micro - Electro - Mechanical Systems) accelerometer, which works by keeping track of the movement of a small mass that is in contact with multiple springs, which are in turn connected to silicon plates. The

mass forms a capacitor with the plates, and the capacitance of that system can be measured. As the mass accelerates in any direction, the capacitance of the mass and the plate in that direction increases as the distance decreases. By tracking the difference in capacitance, the accelerometer can track the acceleration of the sensor in any direction. The gyroscope uses a similar system, and tracks the angular velocity using the capacitance of a mass and plate. Finally, the BNO055 includes a third sensor, a magnetometer, which tracks changes to the magnetic field in relation to the sensor by tracking voltage changes resulting from the Lorentz force on a current-carrying conductor. These 3 different sensors are then combined together natively on the BNO055 using their “sensor fusion” algorithm to act as a Kalman Filter which is a method for getting more accurate data by using the data from other sensors to correct errors in the data from a single sensor. A Kalman filter works by first defining a state variable, which is a vector containing the different variables that need to be tracked. The Kalman Filter then predicts the change in orientation for the next

timestep using a mathematical model, similar to how you would use a given speed and time to calculate the distance traveled. The algorithm then uses sensors to measure the orientation, and compares it to the predicted orientation. Finally, it weighs these two values and combines them according to a given Kalman gain. After the Kalman Filter algorithm calculates the orientation of the drone, these values are then sent to the microcontroller, which will adjust the outputs to the motors so the drone will self-correct if it is out of position. The IMU uses the I2C protocol to communicate with our microcontroller.

For our user control, we went with a prebuilt radio controller, which would send data using radio signals to a radio receiver, which would in turn send data using Pulse Width Modulated (PWM) signals to our microcontroller. We needed 4 different signals to allow the user to control throttle (up/down), pitch (tilt forward/back), roll (tilt left/right), and yaw (turn left/right). This allows the drone to move freely in 3 dimensions.

Once the microcontroller has taken the user inputs and adjusted those values based on the measurements from the IMU, the next step is to turn those into outputs to the motors. These are also PWM signals that will be sent to our 4 Electronic Speed Controllers (ESCs). The main purpose of the ESCs is to act as an interface between the microcontroller and the motor. The microcontroller is a sensitive component that operates at low voltage and low current, while the motors need high voltage and high current. The ESCs bridge this gap by interpreting the PWM signals from the microcontroller into the 3-phase power the motor needs to spin its magnets in a circle.

All of this is controlled by our power system, which can be split into three voltage domains: 3.3V, 5V, and 14.8V. For our 14.8V domain, we simply hook up the battery. Next, we drop that voltage to 5V using a buck converter, which uses a switching regulator to lower the 14.8V from our battery to 5V. After that we use a Low-Dropout regulator to drop the 5 volts to 3 volts.

Lora

To monitor the drone during flight testing and PID tuning, we needed a way to wirelessly transmit sensor data, motor outputs, and system status from the drone back to a ground station. We chose LoRa as the wireless link because it offers long range (over a kilometer in open conditions) at very low transmit power, and uses an inexpensive, easy-to-integrate module, the Reyax RYLR998, that

communicates with the microcontroller over a simple UART connection using AT commands.

However, LoRa has one major drawback for telemetry: the available bandwidth is very limited. The core physical-layer transmission parameters for LoRa are spreading factor, bandwidth, coding rate, and preamble length. These parameters determine the tradeoff between range, robustness, throughput, and airtime. For LoRa, a higher spreading factor lowers the symbol rate, increases receiver sensitivity, and extends communication range, but it also makes each packet take longer to transmit. The same general tradeoff also applies to bandwidth: narrower bandwidth improves RF sensitivity (which increases range) but increases transmission time. Coding rate adds forward error correction, improving robustness against data loss at the cost of extra overhead, while a longer preamble can improve reception reliability against noise but it also adds airtime. [4] [5]

For our settings, we had chosen a spreading factor of 7, 125 kHz bandwidth, 4/5 coding rate, and 8 preamble symbols a single packet takes between 45 and 75 milliseconds to transmit depending on the byte size of the payload. This was determined using the tested throughput in the datasheet [1]. The specific reasons will be elaborated in detail when explaining the challenges of LoRa telemetry. Regardless, designing the telemetry system was therefore mostly an exercise in deciding what data was important enough to send, how often to send it, how to format it efficiently, and ultimately make tradeoffs between range, noise, and transmission time.

One side of the link consists of the STM32 connected to a RYLR998 module by UART and the other side consists of a PC connected to another RYLR998 module by UART, which is connected to a USB-UART TTL device. The module handles all the LoRa physical-layer details internally and exposes a text-based AT command interface to the host. To send a message, the host writes an **AT+SEND** command to the module specifying the destination address, payload length, and ASCII data. When a message is received on the other end, the receiving module emits a line of the following form on its UART:

```
+RCV=<address>,<length>,<data>,<RSSI>,<SNR> [2]
```

The five fields contain the sender's address, the number of bytes in the payload, the actual data we sent, the received signal strength indicator (RSSI, in dBm), and the signal-to-noise ratio (SNR, in dB). For example, a real line received during a test flight looks like:

```
+RCV=1,28,1:1245|0.38,-6.19,5.00,26.8,996.9,-65,8
```

The RSSI and SNR fields are added by the receiving module itself based on the radio conditions when the packet arrived. They give us a real-time measure of link quality independent of the message contents, which is useful for telling whether dropped packets are due to range issues, interference, or other causes.

The LoRa modem provides a transparent byte-level link, but it does not know anything about what kind of data we are sending. To distinguish, for example, IMU data from motor outputs, and to detect packets that get lost in transit, we designed our own message format that lives inside the data field of each LoRa packet:

TYPE:SEQ|PAYLOAD

The type tag is a short identifier for what type of data the packet contains. The sequence number is an integer that increments by one for each message of that type. The payload is a comma-separated list of values whose meaning depends on the type. For example, an IMU reading is sent as:

I:124|0.38,-6.19,5.00,26.8,996.9

This says: message type I (IMU and Barometer), sequence number 124, with five payload values in fixed positions (pitch in degrees, roll in degrees, yaw in degrees, temperature in Celsius, pressure in hPa).

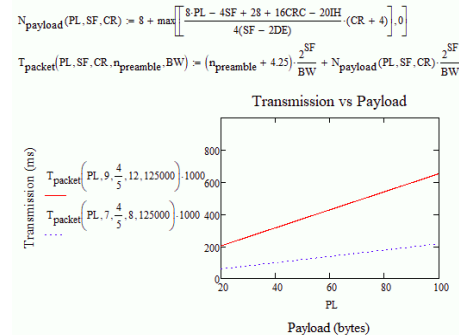
LORA Message Types

Message Type	Meaning	Payload contents
I	IMU and Barometer Outputs	pitch, roll, yaw, temperature, pressure
M	Motor mix PWM Outputs	front-left, front-right, rear-left, rear-right outputs
P	PID terms and components	pitch P, pitch I, pitch D, roll P, roll I, roll D, yaw P, yaw I, yaw D,
B	Battery status	Battery state (NORMAL WARNING CRITICAL)
R	Task rates	flight, tele tx, tele rx, housekeeping
C	CPU usage	flight, tele tx, tele rx, housekeeping, idle
RW	Raw text	Any string
MARI	Special Ping reply	Fixed 88-byte response to a ground-station ping

We had chosen this specific format, because it was relatively easy for the receiving link, which is connected to our python application, to parse and sort data efficiently according to the message type. And by including an incrementing sequence number, the *TYPE:SEQ* effectively a pseudo-UUID, making detection of missing or duplicate packets easier to debug.

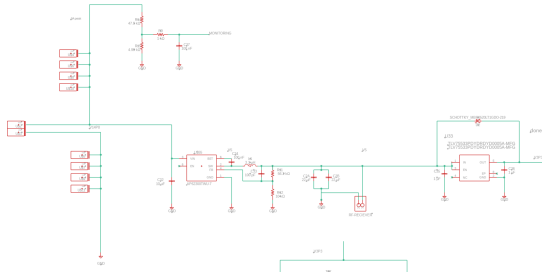
One of the challenges of the LoRa telemetry is finding correct parameters and finding the balance between transmission airtime vs range, robustness, and reliability. Because of the amount of packets being sent per second as well as close range, we ended up sacrificing distance for lowering transmission time, hence why those specific parameters. The exception being Bandwidth and Coding Rate, both of which are not worth changing

for little reduction on time. As shown in the graph below, reducing the Spreading Factor to 7 greatly reduces the transmission time per payload bytes compared to the default of 9.

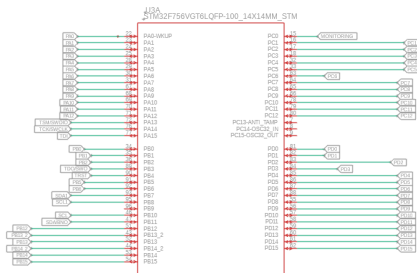


Schematic References:

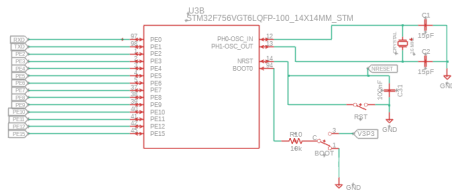
Power System



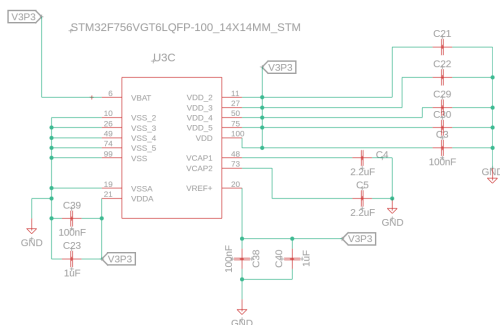
Microcontroller 1



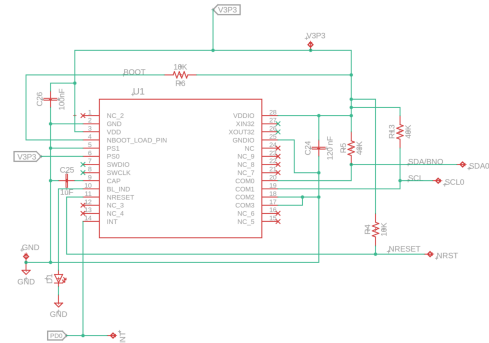
Microcontroller 2



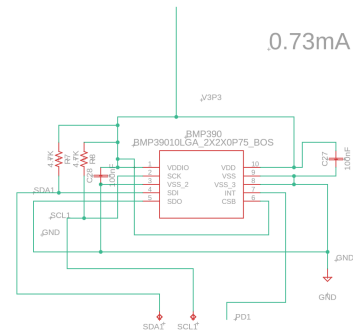
Microcontroller 3



Inertial Measurement Unit



Barometer



Schematic Descriptions

Power System:

The Mari drone Power system is designed to use a single battery to support multiple voltage domains. The system can operate on either a 3S or 4S Lipo battery, with nominal voltages of 11.1V and 14.8V, respectively. Supporting both configurations provides flexibility, but it also requires components to be compatible with the voltage range. Power calculations were performed using the nominal 4S voltage of 14.8 V.

The battery directly powers four ESCs and motors. The motors require the largest share of total power in the system. The lower-voltage components are supported through regulators. After the ESCs and motors, the battery's voltage is stepped down using a buck converter to 5V, then to two separate 3.3V domains using LDOs for sensitive components like the microprocessor and sensors.

To calculate the total system demand, the power of each subsystem was calculated using

$$P = V \times I$$

For the Motors and ESCs, each draws

$I_{motors} = 23.232 A$, so the total current is

$$I_{motors,total} = 4 \times I_{motors}$$

$$I_{motors,total} = 92.928 A$$

The monitoring circuit draws

$I_{monitoring\ circuit} = 0.322 mA$ and is included in the total current calculation.

$$P_{14.8V} = V_{battery} \times (I_{motors,total} + I_{monitoring\ circuit})$$

$$P_{14.8V} = 1375.33 W$$

For the regulated subsystems, the 5V and 3.3V rails are calculated as

$$P_{5V} = 5.0V \times I_{5V,total}$$

$$P_{5V} = 3.364 W$$

$$P_{3.3V} = 3.3V \times I_{3.3V,total}$$

$$P_{3.3VA} = 0.537 W$$

$$P_{3.3VB} = 1.081 W$$

The total system power is then

$$P_{total} = P_{14.8V} + P_{5V} + P_{3.3VA} + P_{3.3VB}$$

$$P_{total} = 1380.318 W$$

The results show that the motors and ESCs accounted for most of the power consumption, compared with the 5V and 3.3V subsystems, which consumed significantly less power but remained critical for system control. To improve voltage stability and reduce noise, LC filtering and bypass capacitors were

included throughout the design. During initial testing, the system was prone to voltage spikes from the battery, which had not been taken into account. This issue was resolved by adding Anti-Spark XT90 connectors to the battery terminals and a bulk capacitor at the PCB input to smooth the input voltage and reduce transient effects. The battery monitoring circuit was also included to measure the voltage and provide feedback via LEDs when voltage levels approached unsafe conditions.

Microcontroller:

As the microcontroller is connected to every other system, most of the pins are used to connect to other systems, or to the header pins we can use to optionally add other systems to our drone. Diagram 1 consists mostly of these types of connections, while Diagram 2 also includes our BOOT circuit and crystal oscillator circuit. Schematic 3 includes the bypass capacitors used for the microcontroller, as well as the main connections to the power system.

While not exactly an error, we misunderstood the function of the BOOT circuit in this schematic. We thought it would turn the microcontroller on/off by cutting it from power, but what it really does is switches one of the operating modes of the microcontroller.

Inertial Measurement Unit:

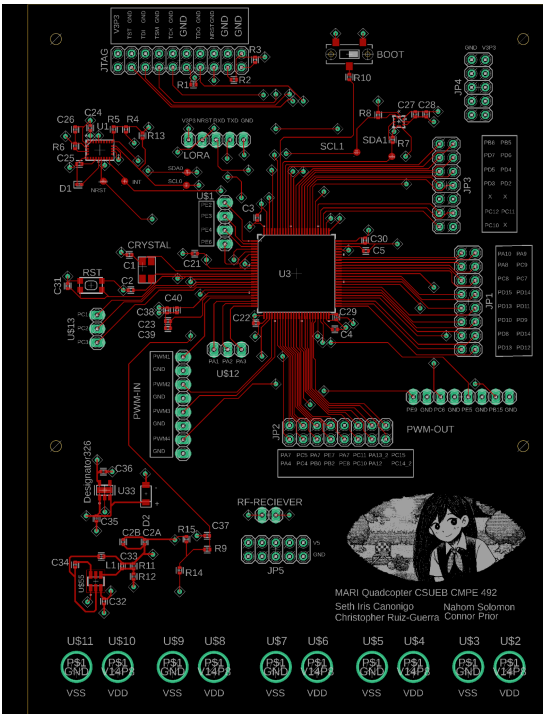
For our inertial measurement unit, we used the recommended schematic from the datasheet for the BNO055. This means connecting all of our I2C data lines to the microcontroller (SDA, SCL and INT), hooking up the power lines to the 3.3V domain, and making sure we had pull-up resistors and bypass capacitors where required.

One error we encountered in this schematic was the PS0 pin was supposed to be set to GND, but was instead set to 3.3V. This ended up changing the “mode” of the communication from I2C to HID over I2C, which made it effectively impossible to communicate with the tools we were using. While we tried to fix the problem with soldering, it ended up being too difficult to fix. We ended up having to use our extra pins on our microcontroller to hook up to an external BNO055 module to keep the use of the IMU.

Barometer:

The barometer schematic is very similar to the IMU schematic, in that we followed the recommended schematic from the datasheet and hooked up the 3 I2C data lines to the microcontroller, as well as connecting it to our 3.3V domain.

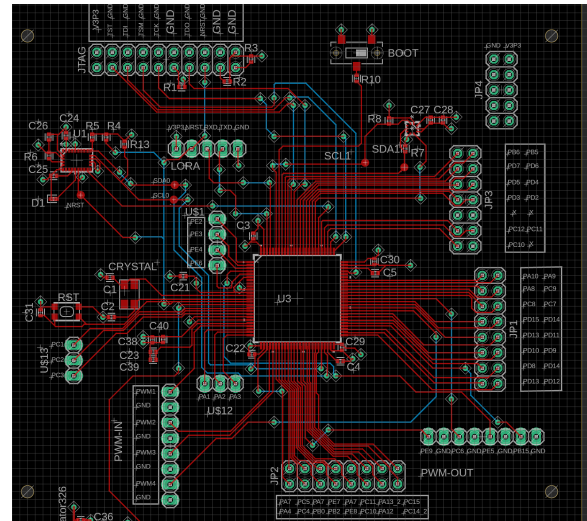
PCB Layout 1



Overall, our PCB holds our power system, microcontroller, IMU, and barometer, and is split according to voltage domain., with the top being the 3.3V domain, the bottom being the 14.8V domain, and a small space in the middle for the 5V domain. Our overall strategy was to keep sensitive components as close to other components as possible, while keeping the voltage domains as clear and separate as possible.

One thing to note in the layout is we used a 4 layer board for our PCB. The red traces are on the top layer, the two middle layers are VDD and ground, and the bottom layer holds the blue traces. Having a bottom layer allows us to route traces much easier, as we can effectively cross traces without shorting them together.

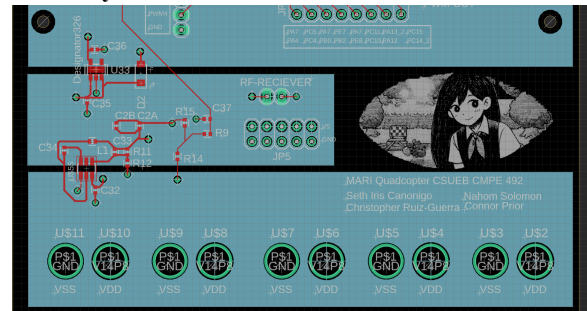
Microcontroller/IMU/Barometer:



The microcontroller is definitely the hub of the board, with connections to almost every component on the PCB. The traces follow the schematic exactly, allowing us to easily connect components on our layout based on the schematic. This layout also shows the many extra pins our microcontroller can use to connect to any external components we might want to add later down the line. We made sure to add identifiers to these pins in the silkscreen, as having to reference the schematic constantly to know which pin was which is too much work. Some pins of note are the JTAG pins on top of the board. These are the pins we connect to our programmer to program our microcontroller.

The IMU and Barometer layout follow the schematic precisely. However, since the pins on the component are under the component, we decided to add test points to almost all of the traces coming out of the IMU, as they would become impossible to access during testing if something went wrong.

Power System:

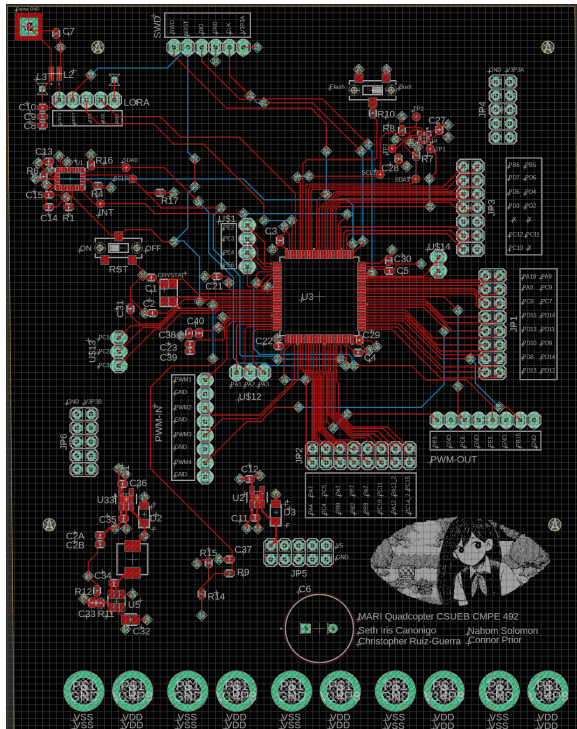


The power system allows us to bridge between clearly defined voltage domains. The bottom part is the 14.8V domain. These throughholes are intended to be used to hold bullet connectors to

connect to 5 different components: the 14.8V battery and the 4 ESCs. The layout clearly shows the 14.8V domain being connected to the 5V domain with the Buck Converter, and the 5V domain being connected to the 3.3V domain with the LDO.

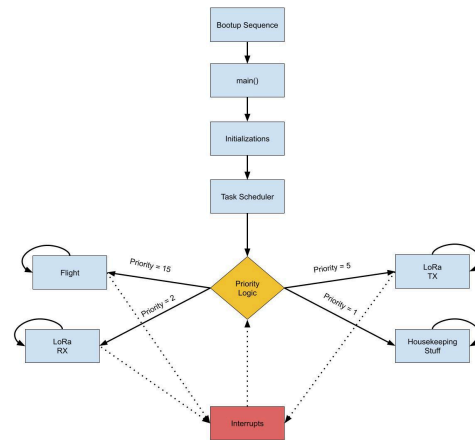
One error we encountered in this layout is the indicators for the Shottky diode. Since we weren't completely aware of the conventions of labelling the anode and cathode of the diode, it ended up being reversed in assembly. This had devastating consequences and resulted in two boards with unusable components. Once we found the problem, it was a simple fix. We just had to switch the diode around and the system worked as intended.

PCB Layout 2



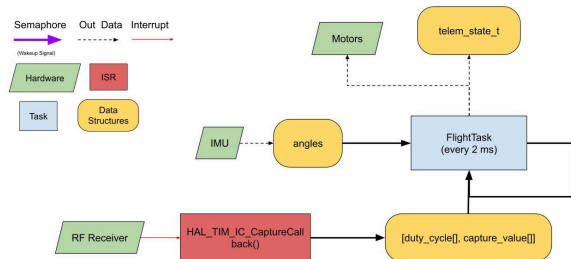
After burning multiple boards and having trouble with the IMU connection, we did a new layout to fix the old problems. The main changes to the new layout are a large capacitor in the 14.8V domain and a larger inductor after the Buck Converter to handle higher voltages. We also cleaned up some of the pins we thought were unnecessary and corrected the IMU connection.

Software



While initially being a single continuous control loop, the software component of our project now is mainly running a real-time operating system (RTOS), specifically FreeRTOS, because the different parts of our software have different priorities with very different timing requirements. For example, the flight control has a hard deadline, being that it can risk crashing the drone if the flight controls do not run fast enough. Other components like telemetry, barometer reading, and battery reading are less important and can even risk a bottleneck, slowing down the flight calculation time. An RTOS lets us guarantee that the time-critical code always runs when it needs to, even if a slower task is in the middle of doing something else. With the RTOS, the overall software is then divided into four main tasks, each running at its own rate and priority: FlightTask, TelemTxTask, TelemRxTask, and HousekeepingTask.

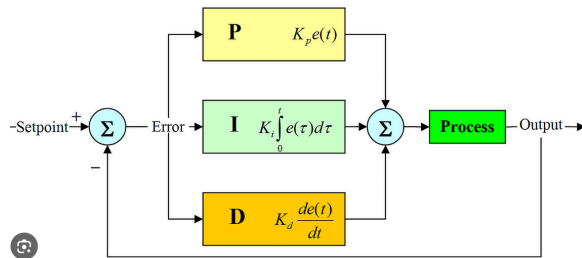
FlightTask:



The most important task is FlightTask, which runs the flight control loop at 500 Hz, meaning it has only 2 milliseconds to read the IMU, compute pitch, roll, and yaw errors, run three PID controllers, apply the mixing equations, and update the four motor PWM outputs. Since it has the highest priority, it preempts whatever lower-priority task is currently running.

First, we have to implement timers to keep track of all the PWM signals we are taking from the radio receiver and outputting to the ESCs. We also initialize the settings on the sensors to make sure they're working properly. We can then move on to the control loop.

To start, we read the user input using timers to determine the duty cycle of the PWM signal. Then, we read data from the registers of the IMU and barometer. After that, we calculate the error between the data from the user input and the IMU. Then its time to use the PID equations.



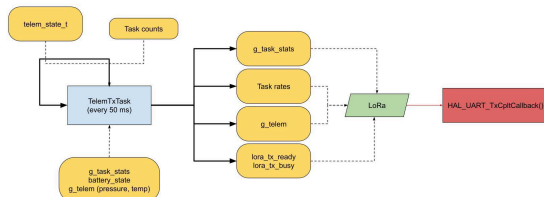
Essentially, the idea of PID equations is to take the error and multiply it by each of the three parts of PID: Proportional, Integral, and Derivative. You then have to manually tweak constant K until your system corrects the error quickly.

Finally, we use mixing equations to turn the intention of the user to outputs to the motors according to whether they want to increase/decrease throttle/roll/pitch/yaw:

Motor1(Front Left) = throttle - roll - pitch + yaw
 Motor2(Front Right) = throttle + roll - pitch - yaw
 Motor3(Rear Right) = throttle + roll + pitch + yaw
 Motor4(Rear Left) = throttles - roll + pitch - yaw

This completes the control loop and we go back to the beginning.

TelemTxTask:

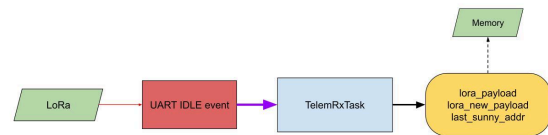


The second most important task is the TelemTxTask. This task decides what to send and when to send it. It runs at 20 Hz, woken every 50 milliseconds by tick polling. Each time the task runs, it does three things: take a snapshot of the latest data from the HousekeepingTask and FlightTask, prepare

it for the message scheduler, and call into the message scheduler to actually send something to UART with the specific *AT+SEND* command. Instead of blindly transmitting every message every cycle, which would massively exceed the LoRa link's capacity, the scheduler implements a round-robin over the message types with two layers of gating: '+OK' check and message type rate limiting.

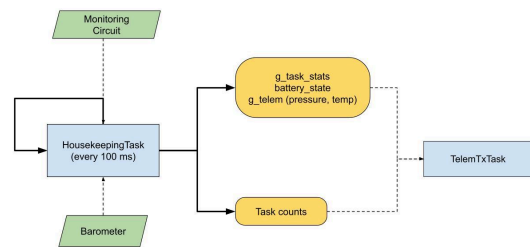
Since every TX signal *AT+SEND* sends back an RX signal *+OK*, the scheduler will always wait until it receives it or wait a one second timeout. This is to prevent overrunning the LoRa. Also since some message types are more important than others, like IMU data vs CPU usage, message types also have priority among each other. The scheduler will send some data with a certain message type more often than others.

TelemRxTask:



The second least important task is TelemRxTask. Unlike the other three tasks, this task is fully interrupt-driven. When the ground station finishes sending a line over UART, a hardware "idle line" interrupt fires, which releases a semaphore that wakes TelemRxTask to parse the message. This keeps the time between a command arriving and the drone responding to it to a few microseconds, instead of polling the UART in software. There is so far only one command available, which is 'pinging' the drone.

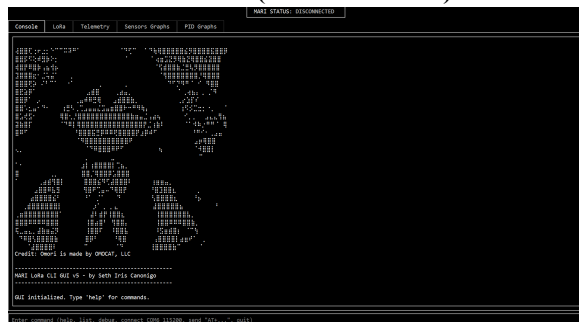
HousekeepingTask



The lowest 'default' task is the HousekeepingTask. Unlike the other three tasks, this task is fully periodic, running every 100 ms when neither FlightTask nor TelemTxTask is running. All this task does is to record the task rates and the CPU usage, run the barometer, run battery monitoring logic, and simply output them to the global variables where the TelemTxTask can access them.

One of the biggest challenges is configuring the speed of polling for the tasks that are both very fast, but also preventing starvation. This was the case for FlightTask, where initially it was given a 1 ms polling period (1 kHz). Unfortunately, because IMU reading plus the PID calculating was over 1 ms, this resulted in FlightTask essentially running forever, starving TelemTxTask and HousekeepingTask, preventing any data from being transmitted. Our solution was to essentially quadruple the I2C bus speed and doubling the period for FlightTask.

MARI LoRa Monitor (Ground Station):



To monitor the drone mid-flight, we developed a desktop Python application using the PyQt6 framework called the ‘MARI LoRa Monitor’. The application is connected to the second RYLR998 through a USB-UART TTL device. It receives the +RCV messages, parses them into readable data according to the LoRa message type table, and updates a live dashboard as shown below.

The screenshot shows the MARI LoRa Monitor application interface with a table of flight data. The table is divided into several sections: Console, LoRa, Telemetry, Sensors Graphs, and PID Graphs. The Telemetry section is currently selected, showing a table of various flight parameters.

Telemetry		Sensors Graphs		PID Graphs	
Pitch P	2.00	Pitch (deg)	82.00	Flight Rate (Hz)	802
Pitch I	7.00	Roll (deg)	14.10	Tail Tx Rate (Hz)	800
Pitch D	0.00	Yaw (deg)	10.50	Tail Rx Rate (Hz)	724
Roll P	7.00	Temperature (°C)	29.0	Housekeeping Rate	100
Roll I	4.00	Pressure (hPa)	980.0	Flight CPU %	86.8
Roll D	2.00	Altitude (m)	117.30	Tail Tx CPU %	87.0
Yaw P	0.00	Battery Status	100%	Tail Rx CPU %	79.0
Yaw I	0.00	Motor 01	1000.00	Housekeeping CPU %	82.0
Yaw D	7.00	Motor 02	1000.00	Idle CPU %	84.0
Pitch PID (Total)	9.00	Motor 03	1000.00		
Roll PID (Total)	14.00	Motor 04	1000.00		
Yaw PID (Total)	21.00	Time			

Results

Overall, while we have come a long way to finishing the drone project, we still have a long way to go to completing it. This is due to the problems we encountered in our power system and the pins we got wrong in the development of the PCB. However, we have still made good progress. The power system is now completely functional and the software is working as intended. While the IMU on the board is

no longer working, we have used an external module that works fine. The overall power consumption of our system ended up being 1091.9W. The total cost of the components on the PCB came to \$45.52. However, in reality each board was about \$100 each to construct. The off-board components added up to around \$330. Therefore, the total cost for an individual drone was about \$430.

Conclusion

Overall, we still have plenty to work on with our drone project. While all of our systems work, we still need to tune the PIDs to get stable flight to help the drone stay in the air. One of the main lessons we learned was how difficult the power system was to get right. Having a good understanding of the fundamentals of circuits will always be very important when designing systems like this drone. We also learned how important step-by-step verification is. It is very hard to debug a system when you’re not sure what the problem is. Finally, we learned how important being able to see data while testing is. It is a lot easier to debug when you can notice patterns in the data and form hypotheses based on it.

Works Cited

- [1] Bosch Sensortec, *BNO055 Intelligent 9-axis absolute orientation sensor*, BST-BNO055-DS000-18, October 2021
- [2] REYAX Technology Co., Ltd., *RYLR998 UART Interface 868/915 MHz LoRa Antenna Transceiver Module Datasheet*, 27-Feb-2025, “Data Throughput Test,” p. 10.
- [3] REYAX Technology Co., Ltd., *LoRa AT Command Guide: Apply for RYLR998, RYLR498*, 7-Aug-2025, “+RCV Show the received data actively,” p. 8.
- [4] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melià-Seguí, and T. Watteyne, “Understanding the limits of LoRaWAN,” *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017, doi: 10.1109/MCOM.2017.1600613.
- [5] Semtech Corp., *SX1272/3/6/7/8 LoRa Modem Design Guide*, application note AN1200.13, Rev. 1, Jul. 2013, sec. 4, “The LoRa Packet Format & Time On Air,” p. 7.